

Tymbolica Rubi

Symbolic integration with genuine Rubi steps

A focused companion to Tymbolica: pass in portable Symbolica Atom payloads, receive an antiderivative or the complete nested rule trace.

Repository · MIT license

1 Use Rubi from Tymbolica

Tymbolica Rubi is a separate package and WebAssembly engine. It deliberately does not parse expressions, declare symbols, or print mathematics. Tymbolica and Tydenso create the versioned Atom payloads it consumes, and either package can continue working with the result.

Import the Rubi package explicitly alongside Tymbolica:

```
#import "@local/tymbolica:0.1.0" as sym
#import "@local/tymbolica-rubi:0.1.0": integrate, integrate-with-steps

#let x = sym.math($x$)
#let f = sym.math($x / (x + 1)$)
#let primitive = integrate(f, x)

$ integral #sym.to-typst(f) dif x = #sym.to-typst(primitive) + C $
```

Evaluating either integration function constructs the fixed bundled module through a pure zero-argument function. There is no loaded flag or call-order dependence: every new instance starts from the same Wizer-preinitialized Rubi tables.

1.1 A worked rule trace

$$\int x(1+x)^{-1} dx = x - \log(1+x) + C$$

Rule 49 : Algebraic expansion

$$\int x(1+x)^{-1} dx = \int 1 - (1+x)^{-1} dx$$

Rule 2009 : Integrate the terms of the sum separately.

$$\int 1 - (1+x)^{-1} dx = \int 1 dx + \int (-1-x)^{-1} dx$$

Rule 24 : Apply the direct antiderivative formula.

$$\int 1 dx = x$$

Rule 16 : Apply the direct antiderivative formula.

$$\int -1(1+x)^{-1} dx = -1 \log(1+x)$$

complete is true when Rubi eliminated every integral. A partial result is still returned when it is false. No integration constant is added. Symbol namespaces, attributes, and portable attachments are retained in the result and in each step payload.

2 Public API

The package exports exactly two public functions.

Function	Result
<code>integrate(expression, variable)</code>	The best-effort antiderivative as portable Atom payload bytes.
<code>integrate-with-steps(expression, variable)</code>	A dictionary containing result, complete, and nested steps.

Both arguments must be portable Tymbolica Atom payload bytes, and `variable` must encode one symbol. Invalid input produces an error instead of being reparsed from its printed form. Thus `math($x$)` is a valid variable, while the annotated Typst content returned directly by `symbol("x")` must first be passed through Tymbolica's `atom` function.

2.1 Step fields

Each step contains:

- `rule`: the Rubi rule number, or none for an auxiliary transformation;
- `depth`: nesting depth of the rewritten integral;
- `description`, `references`, and `source`: Rubi's explanatory metadata; and

- input and output: portable Atom payload bytes for the immediate rewrite.

The array is ordered from the outer rewrite into nested integrals. Use the depth field to indent it, group it, or build a custom explanation layout.

3 Installation from this repository

Install tymbolica and tymbolica-rubi as independent local packages. On Linux, expose the repository root as Tymbolica and its rubi directory as the Rubi package:

```
mkdir -p ~/.local/share/typst/packages/local/tymbolica
mkdir -p ~/.local/share/typst/packages/local/tymbolica-rubi
ln -s /path/to/tymbolica \
    ~/.local/share/typst/packages/local/tymbolica/0.1.0
ln -s /path/to/tymbolica/rubi \
    ~/.local/share/typst/packages/local/tymbolica-rubi/0.1.0
```

The Rubi directory contains its own compressed Wizer-preinitialized engine and its own inflater. Documents that use only Tymbolica do not construct this engine.

Tymbolica Rubi uses Symbolica's integration interface and the Rubi ruleset. Thanks to the Symbolica and Rubi contributors for making the engine possible.